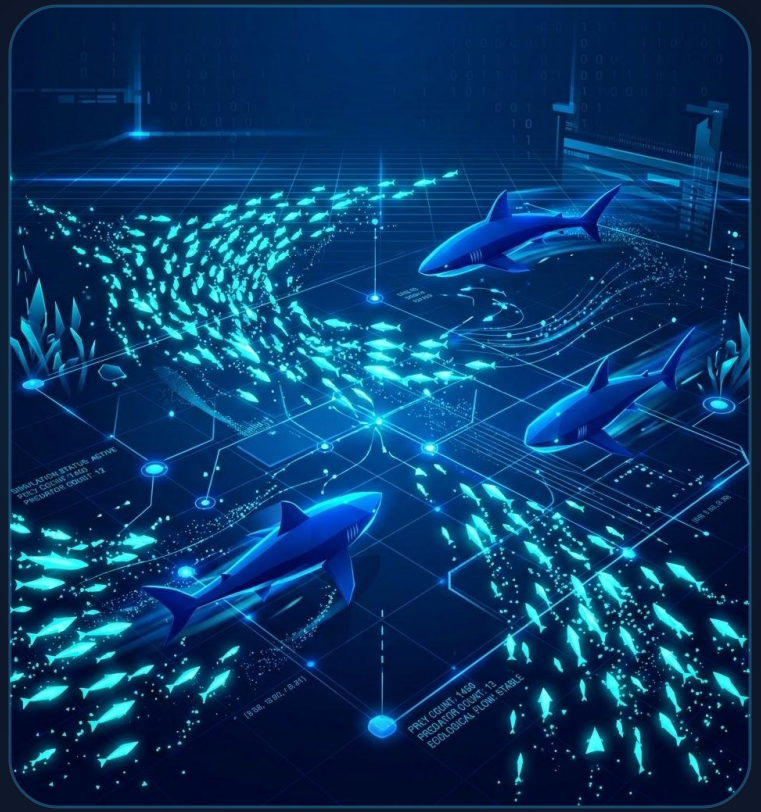
PROJET DE SIMULATION

Wa-Tor

La programmation orientée objet appliquée à un contexte de recherche



Présenté par

Jocelyne Minard, Matisse Guerguet, Romain Ravasini

Simulation & Modélisation

Les bases théoriques

01. Origine & Modélisation (1984)

Wa-Tor est une simulation de dynamique de population modélisée par A.K. Dewdney en 1984.

02. Environnement Toroïdal

Le but de la simulation est de générer un monde toroïdal (en donuts) et de paramétrer le comportement de prédateurs et de proies pour faire un écosystème viable.

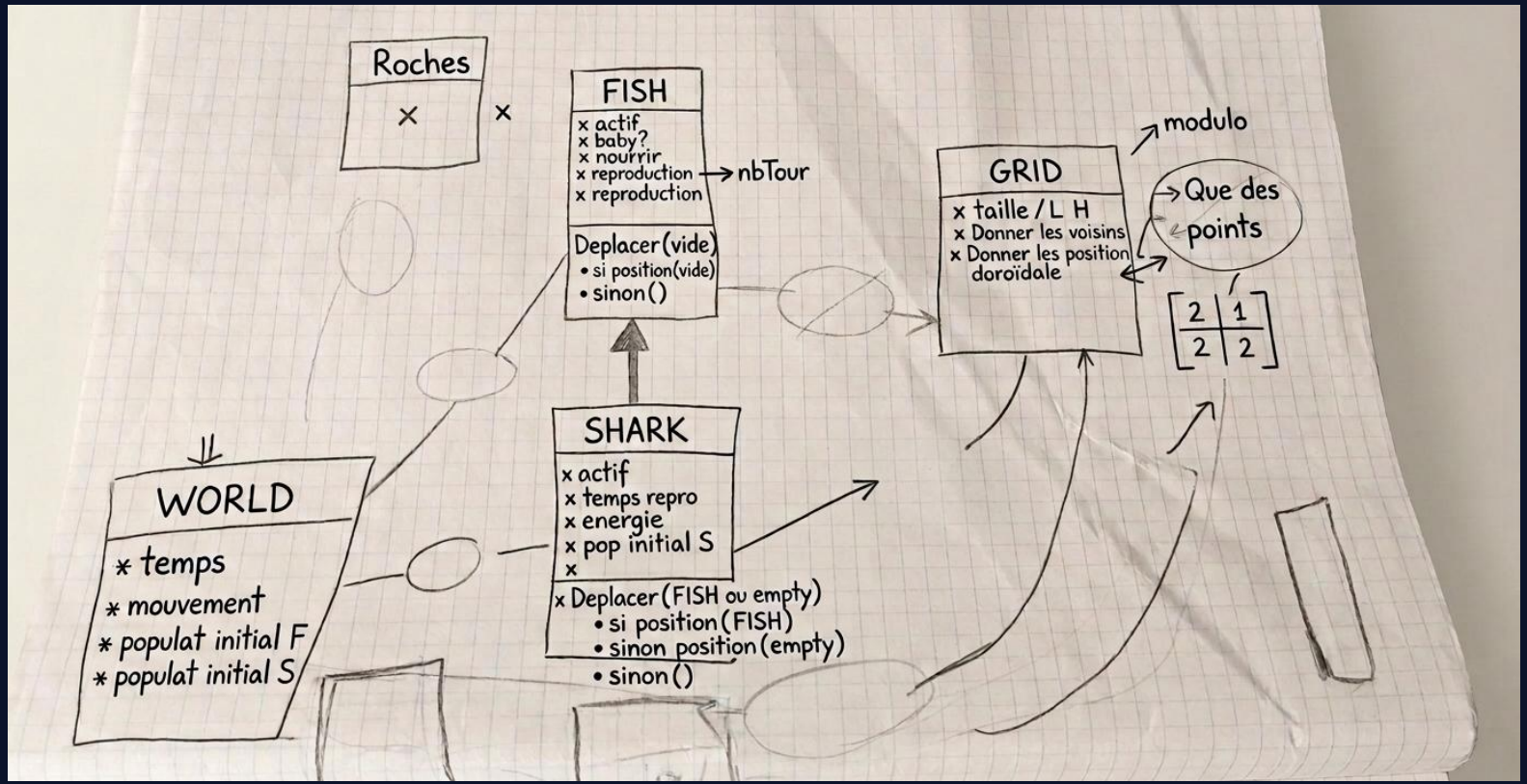
03. Modèle Mathématique

Cette simulation se base sur un modèle mathématique de prédateur et de proie de Lotka-Volterra, visant à rendre le système de prédation en milieu naturel explicable en termes mathématiques.



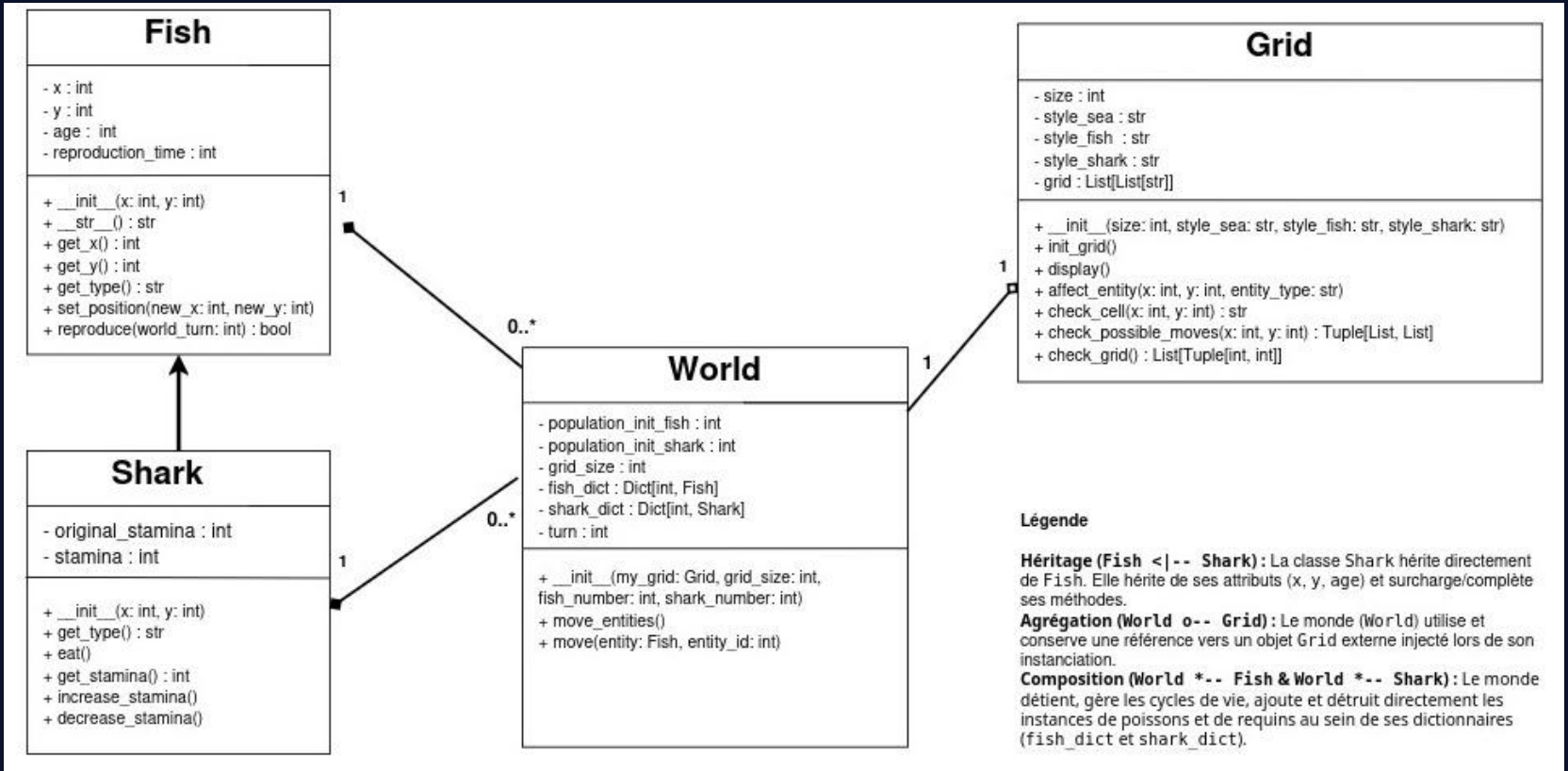
Les origines du projet

Notre UML originel



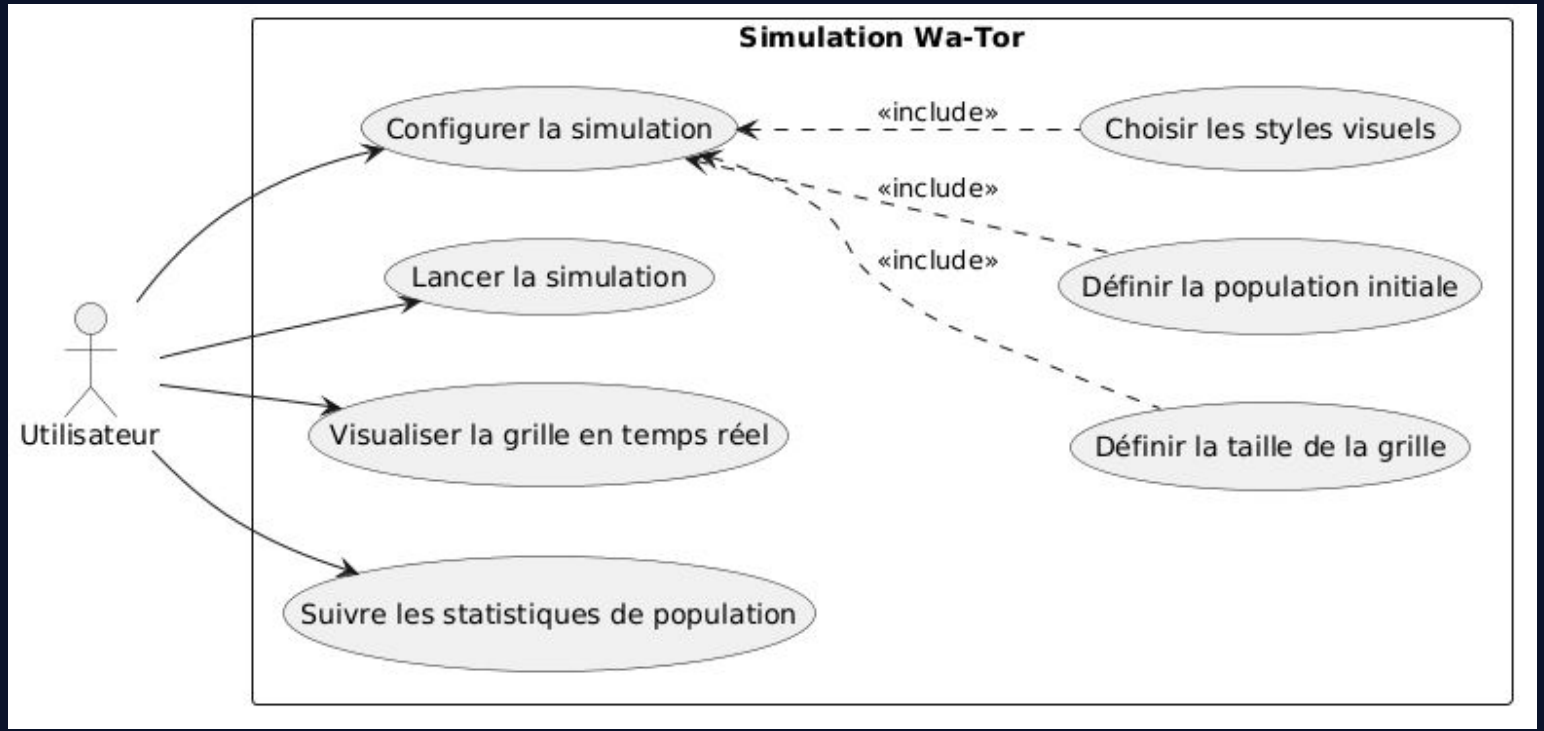
Documents de conception

L'UML suivi



Ce qui est demandé

Quel est notre livrable ?



Nos premières erreurs

Aller trop vite

```
47  -           for cell in adjacent_cells:
48  -               if self.grid[cell[1]][cell[0]] == self.style_sea:
49  -                   empty_cells.append(cell)
50  -               elif self.grid[cell[1]][cell[0]] == self.style_fish:
51  -                   fish_cells.append(cell)
52  +               else:
53  -                   shark_cells.append(cell)
    47  +
    48  +           for y, row in enumerate(adjacent_cells):
    49  +               for x, cell in enumerate(row):
    50  +                   print(cell)
    51  +                   if cell == self.style_sea:
    52  +                       empty_cells.append((x, y))
    53  +                   elif cell == self.style_fish:
    54  +                       fish_cells.append((x, y))
    55  +                   else:
    56  +                       shark_cells.append((x, y))
```

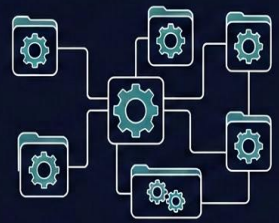
```
31  -           self.stamina == min(self.stamina + 7, self.original_stamina)
    31  +       self.stamina = min(self.stamina + 7, self.original_stamina)
```

Notre UML n'était pas encore finalisé que nous voulions déjà commencé à écrire le code de façon efficace.
Nous nous sommes vite retrouvés à recommencer des classes depuis le début.

Nos premières erreurs

Passer d’une programmation classique à la POO

Programmation Classique vs. Programmation Orientée Objet (POO)

Programmation Classique	Programmation Orientée Objet (POO)
	
01. Approche séquentielle 02. Données et fonctions séparées 03. Structure linéaire et simple	01. Approche modulaire (objets) 02. Encapsulation (données+fonctions) 03. Système complexe et réutilisable

Nos points forts:

- l’encapsulation
- définir les classes et les objets
- le travail de groupe

Nos points à travailler:

- les héritages
- la sécurisation des fonctions
- les doublons

Détails technique

Modifier les dictionnaires sur les boucles.

Boucler sur le dictionnaire et essayer de faire une action dessus en même temps, Python n'aime pas cela

```
    # Find and delete fish from position
    for fid, fobj in self.fish_dict.items():
        if fobj.get_x() == target_x and fobj.get_y() == target_y:
            del self.fish_dict[fid]
    entity.eat()
```

La solution, lors de la boucle, utiliser le dictionnaire en list()

```
# Find and delete fish from position
for fid, fobj in list(self.fish_dict.items()):
    if fobj.get_x() == target_x and fobj.get_y() == target_y:
        del self.fish_dict[fid]
entity.eat()
```

Détails technique

Explication d’une méthode :

```
def move(self, entity, entity_id: int):
```

Cette fonction est le centre névralgique de notre simulation, elle gère le comportement des entités poisson et requin. Nous distinguons les deux types d’entité par un ‘match / case’.

Elle possède en variable, la position, le type de l’entité ainsi que des dictionnaires avec les cellules vides et les cellules contenant des poissons autour de la position de départ. Elle initialise également deux variables dx, dy afin de gérer le déplacement futur.

Dans le cas d’un poisson, nous vérifions si il y a des cases vides disponible autour et dans ce cas, nous regardons si le poisson est en capacité de se reproduire. Si oui, le poisson bouge en laissant une nouvelle entité à sa position initiale et instancie une variable pour préparer son déplacement. Si aucune cellules vides n’est disponible le poisson ne bouge pas et on continue la boucle.

Détails technique

Explication d’une méthode : `def move(self, entity, entity_id: int):`

Dans le cas d’un requin, nous vérifions en priorité si il y a des cellules à coté contenant des poissons, si c’est le cas nous choisissons un poisson à manger avec un `random.choice` et stockons sa position pour s’y rendre. On parcourt ensuite le dictionnaire des poissons pour le supprimer.



Ensuite nous faisons appel aux différentes fonctions du requin pour augmenter son énergie, et la diminuer de 1 suite à son déplacement. Nous avons fait le choix d’augmenter l'énergie de la valeur donnée puis de la décrémenter pour conserver la logique du brief.

Si aucun poisson n’est à côté de lui nous regardons alors les cellules vides pour se déplacer. Et comme pour le poisson selon si il est capable de se reproduire il laisse un autre requin à son emplacement d’origine.

Détails technique

Explication d’une méthode :

```
def move(self, entity, entity_id: int):
```

Notre simulation ne permet pas de manger et se reproduire sur le même tour pour le requin, au vu de l'énoncé : “se déplacer, manger ou se reproduire”

A la fin de la boucle nous regardons l'énergie du requin et s’il doit mourir, dans ce cas nous le supprimons du dictionnaire des requins.

Lorsque l’ensemble des cas est traité si la variable moved a été incrémentée lors des différents cas la boucle gère le mouvement des entités en déplaçant l’entité graphiquement sur la grille.

Détails technique

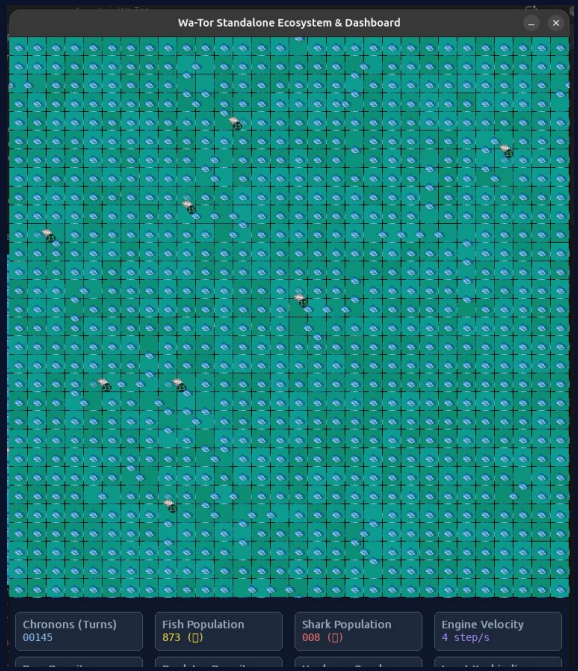
Utiliser une interface graphique (Pygame)

- **Séparation des rôles :** L'interface n'intègre aucune logique de jeu ; elle se contente d'observer et d'afficher l'état envoyé par les classes `World` et `Grid`.
- **Optimisation & Performance :** Pré-rendu en cache des vagues et des emojis (🐟, 🦈) pour tenir les 60 FPS, avec une liste de polices de secours multi-plateformes.
- **Immersion visuelle :** Glissement fluide des entités (interpolation), gestion des bords (tore façon Pac-Man) et fondu progressif des entités mortes.
- **Interactivité complète :** Pause (ESPACE), réglage dynamique de la vitesse, mode plein écran (F11) et tableau de bord des statistiques en temps réel.

Nos réussites

Nos résultats !





Nos réussites

Un travail d’équipe qui compte

- Définitions des objectifs personnels:
  Travail individuel sur les fonctions
- Synchronisation en temps et en heure des fichiers grâce à Git Hub
- De la communication et de l’entraide, personne n’est laissé de côté.
- Une documentation et des annotations claires pour aider à la maintenabilité du projet
- Maintenir un cadre de travail bienveillant et engageant
- Être curieux du travail accompli par ses collègues.

Nos réussites

GitHub



- Environ 15 branches , entre feat et fix
- 15 Pull request
- Remote nettoyer des branches merge
- Local nettoyer des branches supprimées
- Utilisation de git squash mais pas forcément au bon moment
- Un premier merge en ‘main’ avec un numéro de version RC
- Un autre merge en main à la fin avec les dernières corrections, qui donne lieu à une version majeur de la RC
- Création d’un tag git à la version finale
- Utilisation de push force sur les branches personnelle suite au rebase
- Git tag V-1.1.0

Projet Wa-Tor

Merci de votre attention !

Simulation d'écosystème marin & Algorithmes de dynamique des populations